

1. Atividades Práticas - Perfil de Usuário com React.memo

Nesta atividade prática, vamos utilizar o **React.memo** num cenário onde o componente filho recebe várias **props** e o **React.memo** será usado para otimizar a re-renderização, fazendo com que o componente filho não seja re-renderizado se as props relevantes não mudarem.

1.1 Criando o projeto e iniciando o servidor local

Crie um projeto chamado **compras**, digitando o seguinte comando no terminal:

```
npm create vite@latest user-profile -- --template react
```

Este processo de configuração inicial do projeto leva alguns segundos. Ao terminar, o Vite repassa instruções para que você termine de instalar as dependências do seu projeto. Desta forma, digite o seguinte comando para **entrar no diretório** recém criado do nosso projeto:

```
cd user-profile
```

Em seguida, **instale as dependências** necessárias do projeto executando no terminal o seguinte comando:

```
npm install
```

Até aqui, você criou um projeto React usando o Vite e adicionou todas as dependências ao projeto. Você pode, agora, **abrir a aplicação no editor Visual Studio Code**. Para isso, dentro do diretório do projeto digite o seguinte comando no terminal:

```
code .
```

Após a execução deste comando, o Visual Studio Code deverá abrir com a pasta raiz do seu projeto sendo acessada. Em seguida, você inicializará um servidor local e executará o projeto em seu navegador. Digite a seguinte linha de comando no terminal:

```
npm run dev
```

Ao executar esse script, você iniciará um servidor local de desenvolvimento, executará o código do projeto, iniciará um observador que detecta alterações no código e abrirá o projeto em um navegador web. Ele irá rodar a aplicação em modo desenvolvimento em <http://localhost:5173/>.

1.2 Passo 1: Criar a estrutura do projeto

Inicialmente, vamos criar a estrutura do nosso projeto. Logo, dentro do diretório **src**, crie o diretório **components**, que contém os componentes visuais e de interação com o usuário.

1.3 Passo 2: Criar o Componente Filho

Vamos criar um componente filho chamado **UserDetails** que exibirá o nome e a idade do usuário e será memorizado com **React.memo**. Desta forma, dentro do diretório **components**, crie um arquivo chamado **UserDetails.jsx** com o seguinte código:

```
import React from "react";
// Componente memorizado usando React.memo
const UserDetails = React.memo(({ name, age }) => {
  console.log("Renderizando UserDetails..."); // Log para verificar quando é renderizado
  return (
    <div>
      <h3>Detalhes do Usuário:</h3>
      <p>Nome: {name}</p>
      <p>Idade: {age}</p>
    </div>
  );
});
export default UserDetails;
```

Observe neste código que:

- **React.memo:** Memorizamos o componente **UserDetails** usando **React.memo**. Isso significa que o componente só será re-renderizado se as **props** (**name** ou **age**) mudarem.
- **Log no Console:** Usamos **console.log('Renderizando UserDetails...')** para verificar quando o componente é re-renderizado.

1.4 Passo 3: Criar o Componente Pai

O componente **UserProfile** gerencia o estado da idade do usuário e o título da página. Sempre que o botão de título for clicado, o título da página muda, mas o **UserDetails** não deve ser re-renderizado, já que a idade e o nome do usuário não mudam.

Desta forma, dentro do diretório **components**, crie um arquivo chamado **UserProfile.jsx** com o seguinte código:

```
import { useState } from "react";
import UserDetails from "../UserDetails";
function UserProfile() {
  const [age, setAge] = useState(25); // Idade do usuário
  const [title, setTitle] = useState("Perfil do Usuário"); // Título da página

  // Função para alterar a idade
  const incrementAge = () => {
    setAge(age + 1);
  };
  // Função para alterar o título da página
  const updateTitle = () => {
    setTitle(`Perfil Atualizado: ${new Date().toLocaleTimeString()}`);
  };
  return (
    <div>
      <h1>{title}</h1>
      /* Botão para alterar a idade */
      <button onClick={incrementAge}>Aumentar Idade</button>
      /* Botão para atualizar o título */
      <button onClick={updateTitle}>Atualizar Título</button>
      /* Componente filho que exibe os detalhes do usuário */
      <UserDetails name="João Silva" age={age} />
    </div>
  );
}
export default UserProfile;
```

Observe neste exemplo que:

- **useState:** Gerencia dois estados: **age** e **title**.
- **incrementAge:** Incrementa a idade do usuário quando o botão "Aumentar Idade" é clicado.
- **updateTitle:** Atualiza o título da página com a hora atual quando o botão "Atualizar Título" é clicado.
- **Componente UserDetails:** Exibe o nome e a idade do usuário. Ele é memorizado com **React.memo** e só será re-renderizado se o nome ou a idade do usuário mudar.

1.5 Passo 4: Componente Principal

Finalmente, integramos o componente **UserProfile** ao componente principal **App**. Desta forma, altere o arquivo **App.jsx** com o seguinte código:

```
import UserProfile from "../components/UserProfile";
function App() {
  return (
    <div>
      <h1>Exemplo de React.memo com Várias Props</h1>
      <UserProfile />
    </div>
  );
}
export default App;
```

Neste exemplo, observe que:

- **Componente Pai UserProfile:**
 - Gerencia dois estados: **age** (a idade do usuário) e **title** (o título da página).
 - O botão "Aumentar Idade" incrementa a idade do usuário.
 - O botão "Atualizar Título" atualiza o título da página, mas não deve causar a re-renderização do componente **UserDetails**.
- **Componente Filho UserDetails:**
 - Exibe o nome e a idade do usuário.
 - Está memorizado com **React.memo**, o que significa que ele só será re-renderizado se as **props name** ou **age** mudarem. Se apenas o **title** mudar, o componente **UserDetails** não será re-renderizado.

1.6 Passo 5: Testar a aplicação

Agora, execute o código. Inicialmente, você verá que o nome do usuário (João Silva) e a idade (25) serão exibidos na tela do navegador. O título da página será "Perfil do Usuário". No console do Google DevTools você verá a mensagem "Renderizando UserDetails...".

Veja que ao clicar no botão "Aumentar Idade", o valor da idade aumentará e o componente **UserDetails** será re-renderizado, o que será visível no console pelo log "Renderizando UserDetails...", conforme figura abaixo.



Figura 1: Exemplo da nossa aplicação sobre o uso do React.memo após clicar em "Aumentar Idade"

Porém, ao clicar no botão "Atualizar Título", o título da página será atualizado, mas o componente **UserDetails** não será re-renderizado. Você **não verá o log** "Renderizando UserDetails..." no console porque suas props não mudaram.

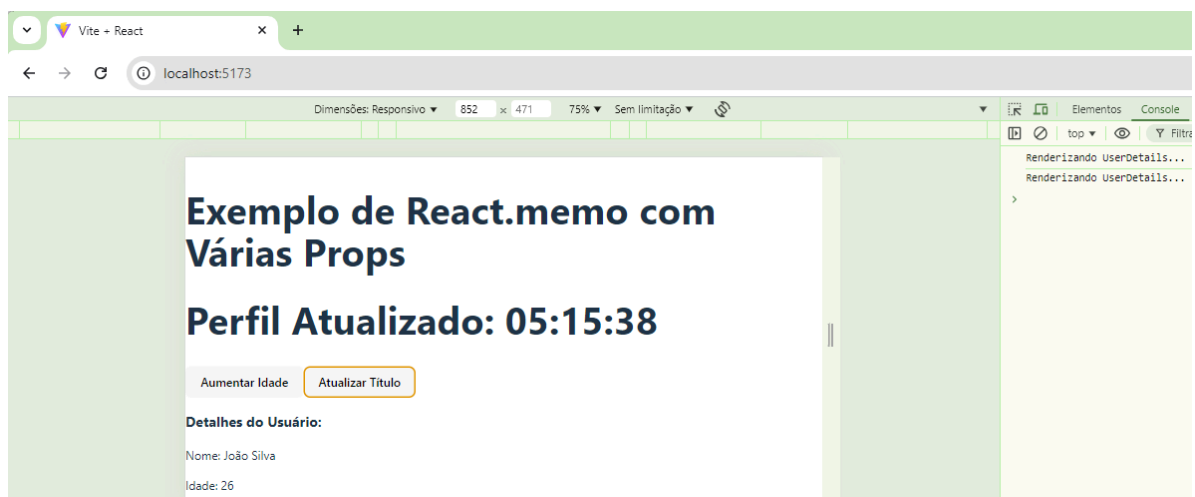


Figura 2: Exemplo da nossa aplicação sobre o uso do React.memo após clicar em "Atualizar Título"

OBS: Retire o uso de **React.memo** do componente UserDetails e veja o seu comportamento no console do Google DevTools.